

```

package demo;
import java.util.*;

public class HashMapDemo {

    public static void main(String[] args) {
        HashMap hm = new HashMap();
        hm.put("ashish", "181");
        hm.put("tushar", "202");
        hm.put("anupam", "303");
        hm.put("rahal", "404");
        hm.put("nishil", "505");
        System.out.println("The batch code for ashish is " + hm.get("ashish"));
        System.out.println("The batch code for tushar is " + hm.get("tushar"));
        System.out.println("The batch code for anupam is " + hm.get("anupam"));
        System.out.println("The batch code for rahal is " + hm.get("rahal"));
        System.out.println("The batch code for nishil is " + hm.get("nishil"));
    }
}

```

Console 10

```

-terminated: HashMapDemo [Java Application] C:\Program Files\Java\jdk-7.0_79\bin\java.exe (May 3, 2013 10:30:20 PM)
The batch code for ashish is 181
The batch code for tushar is 202
The batch code for anupam is 303
The batch code for rahal is 404
The batch code for nishil is 505

```

Figure 5: HashMapCode

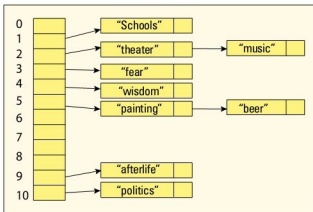


Figure 6: HashMap

storing elements, and arrays are sequentially arranged in the memory. It implements the List interface and extends the `AbstractList` class. It can contain duplicate elements, and it maintains an insertion order. The elements can be accessed using the index notation. This class contains various inbuilt methods, which help in making data retrieval and manipulation easy.

LinkedList class: This class uses a doubly-linked list to store elements. It maintains an insertion order, and it can be used as a list, stack or queue. It actually implements the List and Dequeue interfaces and extends the Abstract class. The `add` and `remove` methods are used to add and delete. The `size` method returns the number of lists in the list.

HashMap: This is used to store key value pairs. It implements the Map interface and is not a part of the collection. HashMap uses the hashing technique to store key value pairs. The key value pair is called a bucket. Each bucket is stored in an array in the HashMap class. The hashing function works on the key and returns an index at which the bucket is stored. There are chances of two keys returning the same index

```

import java.util.*;
import java.util.Set;
import java.util.Iterator;
import java.util.Map;

public class TreeMapDemo {

    public static void main(String[] args) {
        TreeMap<Integer, String> tmap = new TreeMap<Integer, String>();
        //adding elements in TreeMap
        tmap.put(1, "Data1");
        tmap.put(22, "Data2");
        tmap.put(70, "Data3");
        tmap.put(4, "Data4");
        tmap.put(2, "Data5");
        // Display content using Iterator
        Set set = tmap.entrySet();
        Iterator iterator = set.iterator();
        while(iterator.hasNext()) {
            Map.Entry entry = (Map.Entry)iterator.next();
            System.out.println("key is: " + entry.getKey() + " & value is: " + entry.getValue());
        }
    }
}

```

Console 11

```

-terminated: TreeMapDemo [Java Application] C:\Program Files\Java\jdk-7.0_79\bin\java.exe (May 3, 2013 10:35:08 PM)
key is: 1 & value is: Data1
key is: 2 & value is: Data5
key is: 4 & value is: Data4
key is: 22 & value is: Data2
key is: 70 & value is: Data3

```

Figure 7: TreeMapDemo

upon hashing. In such cases, the buckets are stored at the same index but are linked, the same as in a Linked List. Each bucket contains the pointer to the next bucket. The `equals` method is overridden, and the value is fetched when the key matches the key in the bucket. HashMap contains two basic methods, called `get` and `put`, to get the value and put the key value pair in the map.

TreeMap: The TreeMap class implements the Map interface by using a tree. A TreeMap provides an efficient means of storing key/value pairs in sorted order, and allows rapid retrieval. You should note that, unlike a HashMap, a TreeMap guarantees that its elements will be sorted in ascending key order. You can specify the sorting order while creating TreeMap by providing an explicit 'comparator' to TreeMap. Basically, you can create TreeMap in Java in different ways—a TreeMap with a natural sorting order, and TreeMap with custom sorting order by providing the comparator, by copying the sorting order from the other SortedMap, etc.

TreeSet: This provides an implementation of the Set interface that uses a tree for storage. Objects are stored in sorted, ascending order. Access and retrieval times are quite fast, which makes TreeSet an excellent choice when storing large amounts of sorted information that must be found quickly. It contains unique elements only.

To end this discussion, I must say that Java has simplified the use of data structures by providing the ready to use collection framework. For more details, you can visit the Oracle site at <https://docs.oracle.com/javase/tutorial/collections/intro/>.

By Ashish Sinha

The author is a software engineer based in Bengaluru. A software enthusiast by heart, he is passionate about using open source technology and sharing it with the world. He can be reached at ashish.sinha.87@gmail.com